॥ श्री गणेशाय नमः ॥

SQL

Structured Query Language (SQL) is a domain-specific language used to manage data, especially in an rdbms. It is particularly useful in handling structured data, i.e., data incorporating relations among entities and variables.

Sub languages of the structured query language:
- DDL
- DML
- DCL

DDL: Data Definition Language
It is used to define the structure/definition of db tables.
CREATE TABLE, DROP TABLE, ALTER TABLE, PRIMARY KEY, UNIQUE (alt. key), FOREIGN KEY, NON NULL, CHECK.

DML: Data Manipulation Language
It is used to modify data records and access required data from db tables.
INSERT INTO, DELETE FROM, UPDATE...SET, SELECT * FROM table WHERE condition.

DCL: Data Control Language
- Data control for transaction management: used to avoid inconsistencies due to concurrent users.
  ROLLBACK, COMMIT, CHECKPOINT.
- Data control for user authentication: for security
  GRANT ACCESS, REVOKE ACCESS.

SQL queries vs RA queries.

```
SELECT DISTINCT A1, A2, ... An   → projection
FROM R1, R2, ... Rm              → cross product
WHERE P                          → selection
```

$$\Pi_{A_1,\dots,A_n}(\sigma_P(R_1 \times R_2 \times \cdots \times R_n))$$

Q.

Enroll:

sid	cid
s1	c1
s1	c2
s2	c2
s2	c3

course:

cid	Instructor
c1	Korth
c2	Korth
c3	Navathe
c4	Navathe

sid's enrolled in some course by Korth?

→

$$\Pi_{sid}\left(\sigma_{Enroll.cid = course.cid}\left(Enroll \times \sigma_{Instructor = Korth}(course)\right)\right)$$

→ condition is applied before cartesian product.

```
(3) SELECT DISTINCT sid
(1) FROM Enroll, Course
(2) WHERE Enroll.cid = Course.cid
        AND Instructor = 'Korth';
```

→ compilers take care of the optimization.

Basic SQL Clauses

```
SELECT [DISTINCT] A1, ..., An
FROM R1, ..., Rm
[WHERE condition]
[GROUP BY A1, ..., An
    [HAVING condition]]
[ORDER BY A1, ..., An [ASC|DESC]];
```

used to group records

default

Flow of execution:

FROM ⇒ WHERE ⇒ GROUP BY ⇒ HAVING ⇒ SELECT ⇒ DISTINCT ⇒ ORDER BY

FROM: cross product
WHERE: selection

GROUP BY
It is used to group records based on specific attribute values.

```
SELECT X FROM R
GROUP BY X

-- X is a set of attributes
```

```
SELECT X, Y FROM R
GROUP BY X
-- Error: Y is not in the group by clause
```

```
SELECT X, AVG(Y) FROM R
GROUP BY X
-- This is fine, because AVG(Y) is an aggregate function
```

NOTE: These rules vary in the implementations of SQL.

If the GROUP BY clause is used, then accd. to standard SQL, all the attributes in the GROUP BY clause must be in the select clause.
We can use aggregate functions on attributes not in the GROUP BY clause in the SELECT clause.
We aren't allowed to select any other attributes in the SELECT clause.

Aggregate functions

COUNT, MIN, MAX, SUM, AVG
All of these compute aggregations of non-NULL values (NULL values are not considered).

the DBMS creates a row-id attribute. You can access it, but not modify it.

	A	B
1	20	5
2	40	10
3	NULL	15
4	20	20
5	60	25
6	80	30
7	NULL	NULL

```
CREATE TABLE R (
    A INT,
    B INT
);
```

COUNT(*): It gives the # of records.
COUNT(A): It gives the # of non-NULL values of attribute A.

```
SELECT COUNT(*) A1,
       COUNT(A) A2,
       COUNT(DISTINCT A) A3,
FROM R;
-- A1 A2 A3
-- 7  5  4
```

```
SELECT SUM(A) A1,
       SUM(DISTINCT A) A2,
       AVG(A) A3,
       AVG(DISTINCT A) A4
       MIN(A) A5,
       MAX(A) A6
FROM R;
-- A1 A2 A3 A4 A5 A6
-- 220 200 44 40 20 80
```

220/5 200/5

GROUP BY

A	B	C
a1	b1	80
a2	b2	90
NULL	b3	60
a2	b4	60
a1	b1	90
NULL	b3	40
a1	a2	NULL

GROUP BY (A) →

A	B	C
a1	b1	80
a1	b1	90
a1	a2	NULL
NULL	b3	60
NULL	b3	40
a2	b2	90
a2	b4	60

→

A	B	C
a1		
NULL		
a2		

e.g.

```
SELECT A, AVG(C)
FROM R
GROUP BY A
```

A	AVG(C)
a1	85
NULL	50
a2	75

SELECT A FROM R
GROUP BY A
HAVING SOME C > 80 ⇒

A
a_1
a_2

SELECT A FROM R
GROUP BY A
HAVING C > 60

↳ cannot be applied for groups (we need some aggregation like SOME)

Q. Retrieve eid's of employees who get the maximum salary from the relation Emp(eid, sal) where eid is the primary key.

→ $A = U - \bar{A}$

↳ emps with sal less than some other.

$$\pi_{eid}(Emp) - \pi_{eid}\left(Emp \underset{sal<s}{\bowtie} \rho_{I,s}(Emp)\right)$$

```
SELECT eid FROM Emp
EXCEPT
SELECT T1.eid FROM Emp T1, Emp T2
WHERE T1.sal < T2.sal
```

Using the MAX function:

The WHERE clause is tested for each record. The HAVING clause is tested for each record.

```
SELECT DISTINCT eid FROM Emp
WHERE sal = (SELECT MAX(sal) FROM Emp;)
```

NOTE: SELECT MAX(sal) would give sal, we want eid

↳ This is called a subquery (nested non-correlated sub-query).

A subquery can be written in the FROM clause.

Sub-query in FROM clause:

```
SELECT DISTINCT eid
FROM Emp, (SELECT MAX(sal) AS msal FROM Emp Temp)
WHERE Emp.sal = Temp.msal
```

If you use an inner query in the WHERE clause, the result will be a single value, the result is a single record.
If you use an inner query in the FROM clause, the result is a table.

Q. Emp(eid, sal). Retrieve emps with the second highest salary.

First get all rows w/o max. Then get max of those.

$$\rho(Temp,\ \pi_{eid,sal}\left(Emp \underset{sal<s}{\bowtie} \rho_{I,s}(Emp)\right))$$

↳ would give a table Temp w/o the max records

$$\pi_{eid}(Temp) - \pi_{eid}\left(Temp \underset{sal<s}{\bowtie} \rho_{I,s}(Temp)\right)$$

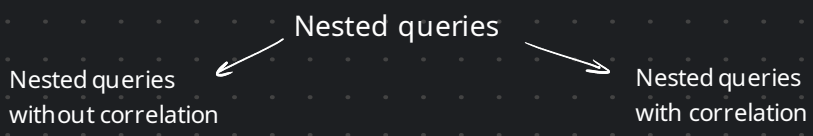
① `SELECT MAX(sal) from Emp` gives max salary

②
```
FROM Emp
WHERE sal < (SELECT MAX(sal) from Emp)
```
} we'll ignore the 2nd maximum

③
```
SELECT MAX(sal)
FROM Emp
WHERE sal < (SELECT MAX(sal) from Emp)
```
} we get the 2nd max salary

But we need the eid

```
SELECT DISTINCT eid FROM Emp
WHERE sal =
  (SELECT MAX(sal)
  FROM Emp
  WHERE sal < (SELECT MAX(sal) from Emp))
```



Nested queries without correlation

E.g. SELECT eid FROM Emp] → outer
WHERE sal = (SELECT MAX(sid) FROM Emp)] → inner

Nothing from the outer query is present in the inner query. The inner query is independent of the outer query.
The inner query executes only once.
The execution order is from bottom to top.

The inner query can be present in the FROM, WHERE, or HAVING clauses.
If the inner query is used in WHERE, then the output will be a constant.
If the inner query is used in HAVING, then the output will be a constant.
If the inner query is used in FROM, then the output is one table.

Nested queries with correlation

```
SELECT * FROM R   ] → outer
WHERE (
        SELECT COUNT(*) FROM S   ] → inner
        WHERE R.A > S.B
        ) < 2
```

A	B
60	40
40	10
30	60
50	40

For a row in R, say (30, 60), every row in S will be checked.

30 > 40 ✗
30 > 10 ✓
30 > 60 ✗
30 > 40 ✗
} count < 2

∴ (30, 60) will be present in the result.
We'll check inner query for each row in the outer table.

result :

A	B
40	10
30	60

The inner query uses attributes of the outer query. The execution flow is top-bottom-top.
The inner query executes once for every record in the outer table.

If the correlation is in the WHERE clause, then the inner query is recomputed for each record of the outer query.
If the correlation is in the HAVING clause, then the inner query is recomputed for each group of the outer query.

Correlated queries are better for complex queries. For simpler queries, non-correlated queries are better.

Functions used in nested queries

IN, NOT IN, ANY, ALL are best suited to nested queries without correlation.
EXISTS, NOT EXISTS are best suited to nested queries with correlation.

```
SELECT * FROM Emp
WHERE sal > (
    SELECT sal FROM Emp
    WHERE depno = 5
)
```

✗ cannot be done. (a value cannot be compared against a list of records)
We need to use the above func"s like ANY, SOME, etc.

IN

X in Y will return true if the X value is a member of the set Y.
The X value must be equal to at least 1 value in Y.
IN can be used in equi-join queries.

50 IN {40, 50, 60, 70} ≡ TRUE

$\Pi_{A,B}(R \bowtie S)$ (natural join) ≡ SELECT * FROM R WHERE R.B IN (SELECT B FROM S)

Q. $R(A,B)$, $S(B,C)$. $\Pi_A(R \underset{R.A=S.B}{\bowtie} S)$

→

```
SELECT DISTINCT A FROM R
WHERE A IN (SELECT B FROM S)
```

Inner query is non-correlated. It'll be executed once first. Then outer query will be run whatever times.

Q. $R(A,B)$ $S(C,D)$ $\Pi_{A,B}(R \underset{R.B=S.D}{\overset{R.A=S.C}{\bowtie}} S)$

```
SELECT DISTINCT A, B FROM R
WHERE (A in SELECT C from S)
    OR (B in SELECT D from S)

SELECT DISTINCT A, B FROM R
WHERE A, B IN SELECT C, D from S
```

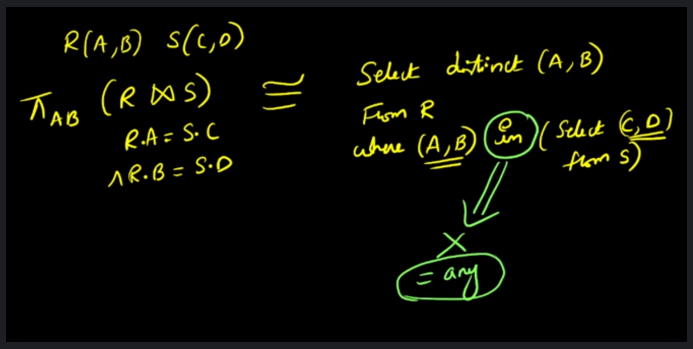
$R(A,..)$ $S(B,..)$

$\Pi_A(R \underset{R.A=S.B}{\bowtie} S)$ ≡

```
SELECT DISTINCT A FROM R
WHERE A IN (SELECT B FROM S);
```

≡

```
SELECT DISTINCT A FROM R
WHERE A = ANY (SELECT B FROM S);
```



ANY, ALL

These are preceded by <, ≤, >, ≥, =, <>.
ANY can be used for conditional join queries.

X < ANY Y will be true if X is less than atleast 1 value in the set Y.

X < ALL Y will be true if X is less than all the values in the set Y.

Q. $R(A,..)$ $S(B,...)$
Get A values of R less than some B values of S.

→ $\Pi_A(R \underset{A<B}{\bowtie} S)$

```
SELECT DISTINCT A FROM R
WHERE R.A < ANY (SELECT B FROM S)
```

≡

```
SELECT DISTINCT R.A from R, S
WHERE R.A < S.B
```

↗ conditional join

Q. Get A values of R less than all B values in S.

→ $\Pi_A(R) - \Pi_A(R \underset{R.A \geq S.B}{\bowtie} S)$

```
SELECT DISTINCT A from R
WHERE A < ALL (SELECT B from S)
```

↗ non-correlated

NOT IN

<> ALL ≡ NOT IN

EXISTS

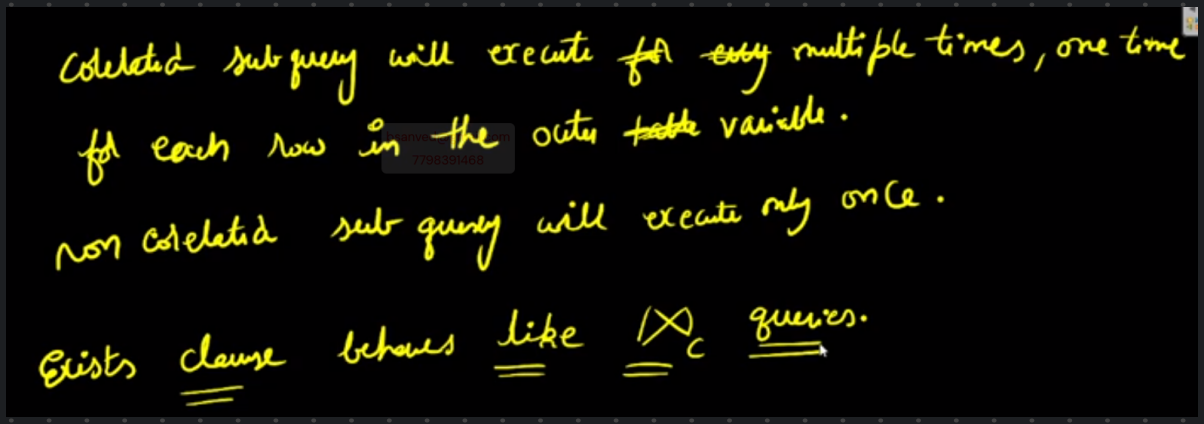
It is used to check if the result of the inner query is empty or not.
EXISTS(inner_query) is TRUE if the result of inner_query has at least one tuple.
EXISTS is also used for conditional joins. It is used in correlated nested queries.

Q. Emp(eid, dno, sal). Retrieve eids of employees whose sal < that of some employee in dno=5.

→

$$\Pi_{eid}\left(Emp \underset{sal<s}{\bowtie} \rho_{I,0,s}\left(\sigma_{dno=5}(Emp)\right)\right)$$

```
SELECT eid FROM Emp as T1
WHERE EXISTS (
    SELECT * FROM Emp as T2
    WHERE T2.salary > T1.salary
    AND T2.dno = 5
)
```



Q. Emp(eid, dno, sal). Retrieve eids whose sal < every other sal in emps of depno 5.

→ $\Pi_{eid}(Emp) - \Pi_{eid}\left(Emp \underset{Emp.sal \geq Temp.sal}{\bowtie} \rho(Temp, \sigma_{dno=5}(Emp))\right)$

```
SELECT eid FROM Emp as T1
WHERE NOT EXISTS (
    SELECT * FROM Emp as T2
    WHERE T1.sal > T2.sal
    AND T2.dno = 5
)
```

Q. R(A, ...), S(B, ...)
Retrieve A values from R that're more than some B values of S.
Give relational algebra query, SQL query with join, nested SQL queries with and without correlation.

→ $\Pi_A(R \underset{R.A > S.B}{\bowtie} S)$

```sql
SELECT R.A FROM R, S
WHERE R.A > S.B;
```
join

```sql
SELECT R.A FROM R
WHERE EXISTS (SELECT * FROM S WHERE R.A > S.B);
```
correlated nesting

```sql
SELECT R.A FROM R
WHERE R.A > ANY (SELECT S.B FROM S);
```
non-correlated nesting

Q. R(A, ...), S(B, ...)
Retrieve A values from R that're more than all B values of S.
Give relational algebra query, SQL query with join, nested SQL queries with and without correlation.

→ $\Pi_A(R) - \Pi_A(R \underset{R.A \leq S.B}{\bowtie} S)$

```sql
SELECT A FROM R
EXCEPT (
    SELECT A FROM R, S
    WHERE R.A <= S.B
)
```
join

```sql
SELECT R.A FROM R
WHERE NOT EXISTS (SELECT * FROM S WHERE S.B >= R.A);
```
with correlation

```sql
SELECT R.A FROM R
WHERE R.A > ALL (SELECT S.B FROM S);
```
without correlation

Q. Emp(eid, sal).
a. Retrieve eids with the minimum salary.

→ min ≡ ≤ every other
≡ U - > some other

$\Pi_{eid}(Emp) - \Pi_{eid}(Emp \underset{Emp.sal > E.sal}{\bowtie} \rho(E, Emp))$

```sql
SELECT eid FROM Emp
WHERE sal = (SELECT MIN(sal) FROM Emp);
```

```sql
SELECT eid FROM Emp as E1
WHERE NOT EXISTS (
    SELECT * FROM Emp as E2
    WHERE E1.sal > E2.sal
);
```

Q. Emp(eid, sal, dno).
a. Retrieve eids of employees who get the min sal for each dept.

$\Pi_{eid}(Emp) - \Pi_{eid}(Emp \underset{\substack{Emp.sal > E.sal \\ Emp.dno = E.dno}}{\bowtie} \rho(E, Emp))$

we could just modify this to add one more condition OR use GROUP BY

```sql
SELECT eid from Emp, (
    SELECT dno, MIN(sal) as min_sal
    FROM Emp
    GROUP BY dno
) as Temp
WHERE Emp.dno = Temp.dno
AND Emp.sal = Temp.min_sal;
```

Division in relational algebra

Q. Enroll(sid, cid), Course(cid, instructor). Retrieve sids of students enrolled in every course taught by Korth.

→ $\pi_{sid}(Enroll) / \pi_{cid}\ \sigma_{instructor = korth}(course)$

There's no division in SQL directly.

If {all Korth cids} - {cids enrolled for by the student}
is an empty set, then the cids enrolled for by the student
contains all Korth cids.

Say Korth cids are korth1 and korth2.
{all Korth cids} = {korth1, korth2}.

For a student, say cids enrolled are {other1, korth1, other2, korth2}
Then {all Korth cids} - {cids enrolled for by the student} =
{korth1, korth2} - {other1, korth1, other2, korth2} = ϕ.
So the student is enrolled in all the courses by Korth.

Similarly, say a student has enrolled in {korth1, other2}.
So not all of the Korth courses.
Then {all Korth cids} - {cids enrolled for by the student} =
{korth1, korth2} - {korth1, other2} = {korth2} ≠ ϕ.
So the student is not enrolled in all korth courses.

```sql
SELECT DISTINCT sid FROM Enroll as T1
WHERE NOT EXISTS (
    SELECT cid from Course WHERE instructor = 'Korth' } all Korth courses
    EXCEPT
    SELECT cid from Enroll as T2 WHERE T1.sid = T2.sid } all courses selected by the student
)
```

Q. WorksFor(eid, pid), Project(pid, pname). Retrieve all eids of employees that work for every project of the name DB.

```sql
SELECT eid FROM WorksFor as T1
WHERE NOT EXISTS (
    SELECT pid from Project WHERE pname = 'DB'
    EXCEPT
    SELECT pid from WorksFor as T2 WHERE T1.eid = T2.eid
)
```